Lecture 18 - Datalog

Setup and Intro

- Goals - to setup Datalog and understand how to support recursive query processing
- A logical perspective:
  - from Codd:
    - equivalence relationship between relational algebra and relational calculus
    - former is operational - construct results using operators such as $\bowtie$, $\cup$, σ
    - latter is declarative - two variants:
      - tuple relational calculus: variable range over tuples
      - domain relational calculus: variable range over domain values
      - like predicate calculus or predicate logic but with more restrictions (so that the queries make sense!)
- Datalog operationalizes the latter
  - A classical database query language with roots in logic
- Central construct is recursion.
- Applications in data integration and exchange, network routing and monitoring, program analysis and security - many of which can be posed as recursive queries/graph traversal queries
- Origins: emerged in the late 1970s–1980s from research on "deductive databases" - a unification of AI and DB work
  - The core idea was to apply logic programming ideas to databases more rigorously than Prolog allowed.
  - Prolog is the prototypical logic programming language
- High-level ideas on query processing:
  - Bottom-up (forward-chaining): start with rules and facts, derive more facts, until convergence
  - Top-down (backward-chaining): start with rules and a desired fact, and then work backwards
  - Will discuss this in more detail in a bit

Formalism

```
r1 reachable(X,Y) :- link(X,Y).
r2 reachable(X,Y) :- link(X,Z), reachable(Z,Y).
query(X,Y) :- reachable(X,Y).
```

- *rules*
  - `A :- B1, B2, . . . , Bn.`, `A` is the head, rest body
- aka `B1` and `B2` and ... and `Bn` implies `A`
- *term* = constant or variable
- *atom / goal* = predicate with terms as arguments
  - `A, B1, ...` are atoms
  - *ground atom* = terms are constants
- *extensional (EDB) predicates* (aka base) - from source data; *intensional database IDB predicates* - inferred (aka derived)
- built in comparison predicates $=, \leq, \neq$ ... - each represented as a special predicate
- *source database instance* is a database instance containing only EDB atoms.
- *active domain* of a database instance is the set of all constants that occur in the database.
- Example above:
  - `link` = EDB, `reachable, query` = IDB
- obvious *range restriction* - any variable in LHS must appear in RHS

Various Semantics

- 1 Model theoretic
  - Based on FOL
  - Datalog rules interpreted as as logical constraints, e.g.,
    - `∀X∀Y(link(X,Y) → reachable(X,Y))`
    - `(∀X∀Y∀Z)(link(X,Y) ∧ reachable(Y,Z) → reachable(X,Z))`
  - Given a source database instance `I` and Datalog program `P,` a model of `P` is a database instance `I'` where `I ⊆ I'` and all rules are satisfied
  - Can show there is a minimal `I'`.
    - Proof: start with a "superset" model formed by taking all active domain of all variables, and adding all possible tuples to all IDBs. Also, intersection of two models is also a model.
- 2. Fixpoint semantics
  - *How* rather than *where*
  - Define the notion of a *immediate consequence* operator `TP` for a Datalog program `P` , which maps database instances to database instances
    - it is monotone: whenever `I ⊆ J`, then `TP (I) ⊆ TP (J)`.
  - Fixpoint: `TP(I) = I`
  - Can show there is a *least fixpoint*
    - Basically, keep applying `TP` until one reaches fixpoint
- 3. Proof theoretic
  - Basically treat each datalog statement as a proof rule.
  - For example:
    1. `link(c,d)` given
    2. `reachable(c,d)` using (1) and the first rule
    3. `link(b,c)` given
    4. `reachable(b,d)` using (2,3) and the second rule
    5. `link(a,b)` given
    6. `reachable(a,d)` using (4,5) and the second rule
  - The set of atoms provable from a source database instance `I` using the rules of `P` is precisely `P(I)`.
  - In the bottom-up (forward-chaining) approach, rules are “factories”, applied repeatedly to derive more and more proofs of atoms.
  - In the top-down (backward-chaining) approach, we start with a candidate output (the goal) and attempt to construct a proof of it, by recursively attempting to prove other atoms from which the goal can be derived.

Negation

- Standard semantics so far: *monotone*
  - if `I ⊆ J`, then `P (I) ⊆ P (J)`.
- Suppose we want to also support negation; datalog variant here is called datalog¬
- Example: compute all pairs of disconnected nodes

```
reachable(X,Y) :- link(X,Y).
reachable(X,Y) :- link(X,Z), reachable(Z,Y).
node(X) :- link(X,Y).
node(Y) :- link(X,Y).
unreachable(X,Y) :- node(X), node(Y), not reachable(X,Y).
```

- A trickier example:

```
p :- not q.
q :- not p.
```

- Harder to reason about
  - has two minimal models `{p}` `{q}`
  - no unique fixpoint
  - from a proof-theoretic approach, not clear how to prove `p` or `q` because neither is being logically entailed as being present due to the rules.
  - but: a database instance containing neither atom fails to satisfy the rules of the program viewed as logical constraints.

One restriction: Semi-positive Datalog

- Negation appears only on EDB predicates
- e.g., computing pairs of nodes which are not directly linked, but are still reachable from one another.

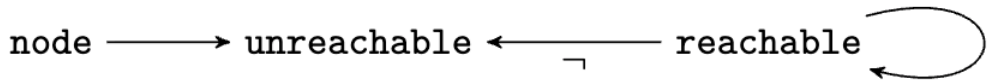
```
reachable(X,Y) :- link(X,Y).
reachable(X,Y) :- link(X,Z), reachable(Z,Y).
indirect(X,Y) :- reachable(X,Y), not link(X,Y).
```

- Extending range restriction to say: every variable must appear in at least one positive atom
  - Q: Why? Constrain the space of possible values it can take
- One way to think about semantics: `not R(X, Y)` is just another EDB that is `R'`
- Thinking about semantics, one can show that for any instance `I` and semi-positive program `P`:
  1. `P` has a unique minimal model `J` such that `J` agrees with `I` on the EDB predicates.
  2. `TP` (immediate consequence op) has a unique least fixpoint `J` such that `J` agrees with `I` on the EDB predicates.
  3. The minimum model and least fixpoints are identical and can be computed in polynomial time by applying `TP` repeatedly to `I`.
- Unfortunately, the proof semantics is harder to reason about because we need to prove the non-existence of a proof tree.

The next less-restrictive restriction: Stratified negation

- Essentially datalog¬ programs that can be written as a sequence `P1, . . . , Pn` of semi-positive datalog programs that can be computed in sequence and "frozen".
- Why is this OK?
  - At each step in the sequence, once we have computed the result of that step, the IDB predicates in the step become EDB predicates (“materialized views”) for the next step.
- Now we need to decide what is a valid *stratification*.
- A valid stratification of `P` is an ordered partition of the IDB predicates in `P` into strata `P1, . . . , Pn` s.t.
  1. If `A :- . . . , B, . . .` is a rule, and `A` is in `Pi` while `B` is in `Pj` , then `i ≥ j`
  2. If `A :- . . . , not B, . . .` is a rule, and `A` is in `Pi` while `B` is in `Pj` , then `i > j`
- Evaluation strategy: first compute the strata, then treat each as semi-positive datalog, with earlier strata's IDBs treated as EDBs.
- Theorem: all stratifications yield equivalent results
- How do we compute a stratification?
  - Construct graph with normal edges (rule 1), and negation edges (rule 2)
  - Compute SCCs, perform topological sort of SCCs

- In any SCC if there is a negation edge, then this is not stratifiable
- Example:



  -
  - Strata: reachable, node, unreachable
- Theorem ((Immerman Vardi, Papadimitriou, Livchack, 1980s): *Any query that can be computed in polynomial time on an ordered database can be expressed by a stratified datalog¬ program.*
  - An ordered database is one equipped with a total order predicate `<`.
  - Note that this result excludes aggregation, etc.
  - In fact, same holds for semi-positive datalog

Aggregation is like negation

```
reachable(X,Y) :- link(X,Y). reachable(X,Y) :- reachable(X,Z), link(Z,Y).
summary(X, count<Y>) :- reachable(X,Y).
```

- Additional range restriction: No variable occurring in an aggregate term in the head of a rule may also occur in an ordinary term in the head of a rule -> this is just like the SQL aggregation restriction
- Unfortunately, recursion through aggregation is problematic, just like negation:

```
p(X) :- q(X).
p(sum<X>) :- p(X).
```

- Q: Why is this problematic?
- Analogous stratified aggregation:
  - Same two rules as negation, plus
  - If `A :- . . . , B, . . .` is a rule in `P` such that `A` contains an aggregate term,
    - and `A` is in stratum `Pi` while `B` is in stratum `Pj` , then `i > j`.
    - (just like negation)
- Computing strata:
  - simply add aggregation as a separate edge, and ensure that there is no cycle with agg edge
- Optimization:

```
path(X,Y,[X,Y],C) :- link(X,Y,C).
path(X,Y,P,C) :- path(X,Z,P1,C1), link(Z,Y,C2), P = [P1,Y], C = C1 + C2.
shortest_path_len(X,Y,min<C>) :- path(X,Y,P,C).
shortest_path(X,Y,P,C) :- shortest_path_len(X,Y,C), path(X,Y,P,C).
```

- Will compute all paths, even those that don't contribute - will not terminate with any cycles
  - paper describes an optimization by eagerly pruning paths

Query Processing

- Example Query: (note, not the same as in book! this is a mistake in the book)

```
r1 reachable(X,Y) :- link(X,Y).
r2 reachable(X,Y) :- link(X, Z), reachable(Z, Y)
query(Y) :- reachable(b,Y).
```

- Let's start with bottom-up approaches
- Naive evaluation:
  - each iteration, compute full immediate consequence
  - Suppose input is `link(a,b), link(b,c), link(c,c), link(c,d)`
    - each item is derived multiple times
- Semi-naive: only new tuples in previous iteration can be used to derive new tuples
  - That is, focus on deltas from previous iteration
  - Suppose: `p :- p1, . . . , pn, q1, . . . , qm.`
  - `δ(p)[i]` - new tuples in iteration `i`
  - `p[i+1] :- (p1[i−1] ∪ δ(p1)[i−1]), . . . , (pn[i−1] ∪ δ(pn)[i−1]), q1, . . . , qm.`
  - Instead, we compute:
    - `Δ(p)[i] :- δ(p1)[i−1], p2[i−1] , . . . , pn[i−1] , q1, . . . , qm.`
    - `Δ(p)[i] :- p1[i], δ(p2)[i−1], p3[i−1], . . . , pn[i−1] , q1, . . . , qm.`
    - ...
    - Collapse all the `Δ(p)[i]` together to derive `δ(p)[i]`, repeat
  - With stratified negation:
    - Treat negated predicates as EDBs - since they would be computed in a previous strata
- Downside of bottom up approaches: evaluate a lot of unnecessary "stuff" even though it isn't needed to answer `reachable(b, Y)`. That said, it is more efficient since computation is done in bulk.
- Top-down approach aka QSQ (Query Sub-Query approach)
  - Approach:
    - start from goal, infer subqueries, and then repeatedly infer other subqueries
    - memoize everything so that you can always recover
    - will terminate because there will be some subqueries that you can compute in entirety as a base case, and then can "roll up"
  - Suppose we have `path(X, Y) :- edge(X, Z), path(Z, Y)`
  - if we wanted to evaluate `path (a, Y)`, then we can unify/bind `X` with `a` - this is *top-down info passing*
  - If after evaluating `edge (a, Z)`, we find the only applicable value of `Z` is `d`, then, `path (d, Y)` is another sub-query - this is *sideways information passing*
    - recall same idea in parallel query processing
  - Basically we record all the subqueries we want to compute, e.g., `path (a, Y)`, `path (d, Y)`, etc. and keep applying rules for each one - and memoize anything we've found.
  - Basic recipe
    - A query arrives with some bound arguments → **top-down passing** determines which subgoal instances to evaluate.
    - Within a rule body, evaluating each subgoal produces new bindings → **sideways passing** propagates them to the next subgoal.
    - The results bubble back up, and memoization (the "table" of already-evaluated subqueries) ensures each distinct subquery is solved only once.
  - Main benefit, you only evaluate the subqueries that are valuable. But downside is that computation isn't done in bulk

Magic Sets:

- Goal: bring can we bring the benefits of Q/SQ to bottom up evaluation
  - *simulating top-down control in a bottom-up engine*

- Main drawback of bottom-up is that it computes the *entire* least fixpoint regardless of what the query actually asks.
- For `path`, that means computing all pairs `(X,Y)` reachable in the entire graph, even if you only care about what's reachable from node `b`.
- Magic sets rewrites the program so that bottom-up naturally restricts itself to the relevant portion of the search space, basically the same as top-down would have

```
reachable(X,Y) :- link(X,Y).
reachable(X,Y) :- link(X, Z), reachable(Z, Y)
reachable(b,Y)?
```

Step 1: Adorn the Program

- Label each argument as `b` (bound) or `f` (free), based on the binding information available when that predicate is called.
- Starting from the query `reachable_bf(a, Y)`, we propagate adornments through the rules:
- `reachable_bf(X, Y) :- link_bf(X, Y).`
  - `X` is bound (from the head's `b`), `Y` is free.
- `reachable_bf(X, Y) :- link_bf(X, Z), reachable_bf(Z, Y).`
  - `X` is bound → this binds `Z` via sideways passing
  - Now `Z` is bound when we call `reachable(Z, Y)`
- adorned program:

```
reachable_bf(X,Y) :- link_bf(X,Y).
reachable_bf(X,Y) :- link_bf(X, Z), reachable_bf(Z, Y)
```

- The adornment analysis is essentially a static simulation of the top-down call patterns that would arise at runtime.

Step 2: Generate Magic Rules (aka demand propagation)

- For each adorned predicate, create a *magic predicate* `magic_p_adornment` whose arguments are the the *bound* arguments of that adornment
  - Basically represents the set of values for which `p` would actually be *called* in the top-down execution
- For example, for `reachable_bf`, bound arg is `X` so `magic_reachable_bf(X)` tracks which first-arguments are demanded.
- From query itself we have `magic_reachable_bf(b).`
- Magic propagation:
  - From this rule: `reachable_bf(X,Y) :- link_bf(X, Z), reachable_bf(Z, Y)`
  - Starting from LHS, we expand the set of "top-down" reachable bound arguments to the RHS
  - if we're asked about `reachable` starting from `X`, and there's an edge `X→Z`, then we'll also need `reachable` starting from `Z`.
  - Thus: `magic_reachable_bf(Z) :- magic_reachable_bf(X), link(X, Z).`
  - The recursive call `reachable_bf(Z, Y)` has `Z` bound. To call `reachable_bf(Z, ?)`, we need `magic_reachable_bf(Z)`. When is `Z` available? When `magic_reachable_bf(X)` holds (we're working on `X`) *and* `link(X, Z)` produces a `Z`.

Step 3: Restrict the original rules

- to those only reachable through top-down exploration, i.e., the magic predicate holds for bound arguments.
- So:

```
reachable_bf(X,Y) :- magic_reachable_bf(X), link_bf(X,Y).
reachable_bf(X,Y) :- magic_reachable_bf(X), link_bf(X,Z), reachable_bf(Z, Y)
```

All together now

```
magic_reachable_bf(b).
magic_reachable_bf(Z) :- magic_reachable_bf(X), link(X, Z).
reachable_bf(X,Y) :- magic_reachable_bf(X), link_bf(X,Y).
reachable_bf(X,Y) :- magic_reachable_bf(X), link_bf(X,Z), reachable_bf(Z, Y)
```

- Magic predicates essentially restrict calls to those that the top-down evaluator would have made.
- Demand propagation rules mirror sideways information passing of the top-down model:
  - just as top-down passes `Z` laterally from `link(X,Z)` into `reachable(Z,Y)`, the magic rule derives `magic(Z)` from `magic(X)` and `reachable(X,Z)`.
- Rewritten program has two separate sets of rules:
  1. **Demand propagation:** the magic rules compute which values are relevant.
  2. **Fact derivation:** the restricted rules derive only the relevant facts.
- Extensions to multiple adornments:
  - You get multiple adorned copies, each with their own magic predicates and rules, non-trivial
    - Same predicate can appear with different adornment patterns in the same program
      - (e.g., `reachable_bf` and `reachable_bb`), each generating its own magic predicate and propagation rules.
    - Adornment propagation analysis is essentially a dataflow analysis over the program's rule graph.
      - If a predicate appears in multiple rules with different binding patterns, you may need to take the join or union of magic predicates depending on how the calls arise.